

Topic 8.5

8.5: More about Exceptions

Functions may be written to acknowledge which exceptions they may throw. This is known as exception specification. It is not mandatory for a function to specify an exception in order for the function to throw the exception. For example:

```
int myfunc(int x, float y) throw (someException,  
                                anotherException);
```

The following form means the **function yourfunc** *cannot* throw an exception:

```
float yourfunc( double z ) throw ();
```

If a function specifies exceptions and then throws an exception not specified, the function `unexpected()` is invoked. Actually, `unexpected()` is a pointer to a function which you may specify by invoking `set_unexpected()` and indicating which function to call in this situation.

Similarly, if an exception is thrown but there is no handler for it, the exception is said to be *unhandled* or *uncaught*. In this situation, the function specified by `terminate()` is invoked. You can set this function using `set_terminate()`.

An exception may be rethrown to a higher level handler:

```
catch ( /* any legal exception declaration */ )  
{  
    ...           // Take any required action.  
    throw;       // Rethrow the same exact exception.  
}
```

In general, do not throw exceptions in constructors. The object is only partially built at that point (remember, we're constructing it!), and the destructor for the object will not be invoked. So, if an error occurs in a constructor, it is generally better to undo whatever the constructor has done to that point directly in the constructor.

Because throwing exceptions causes destructors to be called, it is unwise to throw exceptions within a destructor. Doing so might cause a second exception to be thrown while searching or unwinding the stack for a previously thrown exception. More specifically, do not allow an exception caused during execution of a destructor to "escape" the destructor. Doing so will cause a call to `terminate()`.

Catching exceptions by reference is a good idea, because it keeps a derived exception in a hierarchy from being "sliced" to the base class matching the catch class.

[Back to Module 8 \(https://onlinelearning.berkeley.edu/courses/2291660/pages/module-8\)](https://onlinelearning.berkeley.edu/courses/2291660/pages/module-8)